

CoreFinity Adaptive Training Controller

Research vision, architecture, public library, UI, evaluation plan, and launch specification

Purpose. This document is a build brief for an engineering-capable AI such as Claude. It defines what CoreFinity wants to research and build, the architecture, public library, UI, evaluation methodology, and launch plan. It deliberately treats the core idea as a hypothesis to test—not as a proven breakthrough.

1. THE CORE IDEA

Traditional model training usually follows a predetermined recipe: batches are selected, gradients are calculated, parameters are updated, and the process repeats. CoreFinity wants to investigate **closed-loop adaptive training**: the training system observes what is happening, evaluates the consequences of previous interventions, and decides what to do next.

The central research question is:

Can an intelligent controller learn how to allocate the next unit of training compute so that a model reaches a target capability with less compute, reaches higher quality for the same compute, or potentially both?

Jev is an initial decision-making component, not the research contribution by itself. Deterministic software should handle metrics, execution, safety constraints, validation, budgets, and logging. The long-term goal is a reusable training architecture that supports different controllers.

2. FROM FIXED TRAINING TO CLOSED-LOOP TRAINING

FIXED: Dataset → Epoch 1 → Epoch 2 → Epoch 3 → ... → Done
ADAPTIVE: Training state → Controller → Action → Training interval ↑ ↓ Evaluation ← Outcome ←

The controller may decide which data to emphasize, which data to skip, how to change batch composition, whether to alter learning rate, whether to repeat difficult examples, where to allocate more compute, which experiment branch to continue, and when to stop spending compute.

The important shift is from *repeat the recipe* to *observe → hypothesize → intervene → measure → learn → intervene again*.

3. WHAT THE CONTROLLER CAN CONTROL

- **Data:** select, skip, down-weight, repeat, or increase exposure to defined data groups.
- **Batch composition:** change the mixture of easy, medium, difficult, domain-specific, or failure-focused examples.
- **Optimization:** make bounded learning-rate or related schedule changes.
- **Compute allocation:** give more budget to promising branches and terminate low-value branches.
- **Evaluation:** trigger evaluations when meaningful changes or uncertainty are detected.
- **Early stopping:** stop a branch when additional compute is unlikely to justify its cost.
- **Experiment branching:** create controlled alternative training strategies and compare them.

Later versions can investigate optimizer selection, batch size, precision, regularization, curriculum ordering, and component-specific allocation. These should not all be automated in version one.

4. THE MEMORY / EXPERIENCE LAYER

Jev does not magically remember an entire training run. CoreFinity should explicitly maintain the memory around the controller. Every intervention becomes an experience record.

```
STATE: validation loss 0.67, accuracy 81.3%, LR 2e-4, difficult-data exposure 20% ACTION: increase
difficult-data exposure to 35% OUTCOME: accuracy 82.1%, validation loss -0.03, compute +4% ASSESSMENT:
positive NEXT STATE: recorded for future decisions
```

The controller can then ask: *what happened when we tried something similar under a similar state?* The memory must remain inspectable. It should also record enough context to avoid treating one noisy improvement as proof of causality.

Store state snapshots, actions, outcomes, evaluation results, budgets, seeds, checkpoints, and experiment metadata. Where possible, distinguish controlled comparisons from simple before/after observations.

5. SYSTEM ARCHITECTURE

```
Dataset / Model / Config ↓ Training Executor ↓ Metrics + Gradient Stats ↓ State Builder / Memory ↓
Decision Controller ↓ Action Validator ↓ Next Training Interval ■
```

Core components:

- **Trainer Adapter:** integrates with the training stack without forcing users to rewrite their trainer.
- **State Builder:** converts raw metrics and history into a controller-readable state.
- **Experience Store:** stores states, actions, outcomes, evaluations, and experiment metadata.
- **Controller Interface:** pluggable interface for Jev, rules, learned policies, or future controllers.
- **Action Validator:** deterministic guardrails that reject invalid, unsafe, or out-of-budget actions.
- **Experiment Manager:** tracks branches, budgets, checkpoints, configurations, and results.
- **Evaluator:** runs predefined evaluation suites and produces comparable metrics.
- **Observability:** logs every decision, evidence, expected effect, and actual outcome.

6. JEV'S ROLE

Jev should be treated as an initial controller candidate. It receives a structured representation of the current state, recent experience, available actions, and constraints, then returns a typed proposed intervention.

```
CURRENT STATE + HISTORY + AVAILABLE ACTIONS + BUDGET ↓ CONTROLLER ↓ STRUCTURED ACTION + CONFIDENCE +
OPTIONAL RATIONALE ↓ ACTION VALIDATOR
```

Free-form output must never be executed directly. The action validator enforces the experiment's allowed ranges and safety rules. The framework must also support non-Jev baselines so we can measure whether the controller actually contributes value.

7. PUBLIC LIBRARY

The project should become a public developer/research library, not a private Jev demo. Users should be able to wrap an existing training job with adaptive control while retaining their normal model and dataset tooling.

Illustrative API:

```
from corefinit adaptive import AdaptiveTrainer, Controller
trainer = AdaptiveTrainer( model=model,
dataset=dataset, controller=Controller.jev(api_key=JEV_API_KEY), budget={"gpu_hours": 10} )
trainer.fit() trainer.report()
```

The exact API can evolve. The stable conceptual abstraction should be **trainer + state + controller + actions + evaluator + budget**. Controllers must be vendor-neutral so Jev can be replaced without rewriting the training system.

Suggested repository structure:

```
corefinit-adaptive/ ■■■ corefinit-adaptive/ ■ ■■■ trainer/ ■■■ controllers/ ■ ■■■ state/ ■■■
memory/ ■ ■■■ actions/ ■■■ evaluation/ ■ ■■■ experiments/ ■■■ policies/ ■ ■■■ integrations/ ■■■
```

8. UI / DASHBOARD

The UI should explain *why* the training strategy changed, not merely show generic monitoring charts. It should visually communicate: **observe** → **decide** → **act** → **measure** → **learn**. Use CoreFinity's dark charcoal, white, and restrained deep-blue visual system; avoid generic neon AI dashboards.

- **Run Overview:** current quality, compute consumed, target metric, and remaining budget.
- **Training Timeline:** chronological controller decisions and outcomes.
- **Decision Inspector:** state snapshot, proposed action, confidence, constraints, and observed result.
- **Data Allocation:** how training exposure changed over time.
- **Experiment Tree:** competing branches and where compute was allocated.
- **Evaluation:** quality curves plus efficiency metrics and baseline comparisons.
- **Experience:** searchable history of interventions and outcomes.
- **Budget:** GPU-hours, tokens, steps, wall-clock time, and branch allocation.

9. RESEARCH HYPOTHESES

- H1: Adaptive data allocation can reach a target evaluation score using less training compute than a fixed schedule.
- H2: Adaptive intervention selection can improve evaluation quality under the same compute budget.
- H3: Explicit experience history can improve subsequent controller decisions versus a stateless controller.
- H4: Confidence-aware intervention policies can reduce harmful or low-value changes.
- H5: The benefit depends on model size, dataset characteristics, and task difficulty; the framework must discover where adaptive control helps and where it does not.

10. EVALUATION MUST BE RIGOROUS

This is the credibility-critical part. We must not announce a breakthrough from a single run. Compare multiple controller types under controlled conditions.

- **Fixed baseline:** standard predetermined training recipe.
- **Rule-based adaptive baseline:** simple deterministic thresholds.
- **Stateless controller:** current state but limited historical experience.
- **History-aware controller:** current state plus prior interventions/outcomes.
- **Jev controller:** the initial structured-decision implementation.

Measure final quality, quality at fixed compute budgets, compute required to reach fixed quality targets, GPU-hours, wall-clock time, tokens, steps, controller interventions, positive/neutral/negative intervention outcomes, branch termination efficiency, random-seed variance, and controller overhead.

The key visualization should be a **quality-versus-compute frontier**. A meaningful result is not merely that one run finished faster; it is that the adaptive method consistently changes the trade-off between quality and compute under controlled comparisons.

11. FIRST EXPERIMENT

Do not begin with a frontier-scale model. Begin with a small, reproducible task where many controlled experiments are affordable.

- Choose a small open model and manageable task/dataset.
- Implement a fixed baseline.
- Implement the adaptive framework without an intelligent controller.
- Add a simple rule-based controller.
- Add Jev behind the same controller interface.
- Run repeated experiments with fixed seeds and controlled budgets.
- Determine whether useful intervention patterns are reproducible.
- Scale only if the earlier evidence supports scaling.

The first milestone is not “build a smarter LLM.” It is: prove that adaptive control can make a measurable training decision that consistently improves compute efficiency or quality under controlled conditions.

12. FAILURE MODES

- **Reward hacking:** the controller improves a metric while harming real capability.
- **Noise mistaken for causality:** an improvement occurs after an intervention but was unrelated.
- **Oscillation:** the controller repeatedly increases and decreases a parameter.
- **Evaluation overfitting:** training strategy optimizes the benchmark rather than general capability.
- **Controller overhead:** decision cost exceeds the compute saved.
- **History explosion:** too much memory makes decisions expensive or noisy.
- **Unbounded action space:** too many possible interventions make attribution difficult.
- **Vendor dependence:** the framework becomes inseparable from Jev.
- **False scaling assumptions:** small-model gains fail to transfer to larger workloads.

13. GUARDRAILS

- Hard compute budget.
- Minimum/maximum learning-rate bounds.
- Maximum dataset reweighting.
- Explicit allowed-action list.
- Checkpoint before major interventions.
- Automatic rollback after defined regressions.
- Evaluation gate before branch promotion.
- Complete audit log.

14. PRODUCT / RESEARCH POSITIONING

Do not position it as: “Jev makes model training smarter.”

Position it as: “CoreFinity is investigating whether a closed-loop decision controller can learn where training compute should be spent. Jev is one controller we are evaluating.”

This keeps the project valuable if the best controller eventually turns out to be another model, a learned policy, or a hybrid algorithm.

15. LAUNCH PACKAGE

One research effort should generate a complete public technical footprint:

- Open-source Python library.
- GitHub repository with reproducible examples.
- Research report / technical paper.
- Benchmark configurations and experiment scripts.
- Baseline and adaptive training implementations.
- Interactive UI/dashboard.
- Architecture diagram and benchmark charts.
- Technical launch article.
- CoreFinity Labs LinkedIn announcement.
- Short demo showing the controller changing a training run.

The repository should separate framework code, research experiments, and examples. Installation of the library should not require the entire research environment.

16. LONG-TERM ROADMAP

Phase	Goal
0 — Research design	Formalize state, action space, value/reward definition, baselines, and experiment protocol.
1 — Prototype	Build trainer adapter, state builder, memory, action validator, and fixed/rule controllers.
2 — Jev integration	Add Jev as a pluggable structured-decision controller.
3 — Evaluation	Run controlled experiments and establish whether gains are reproducible.
4 — Public library	Stable APIs, docs, examples, tests, packaging, versioning.
5 — UI	Dashboard for decisions, experiments, budgets, and evaluation.
6 — Research expansion	Learned controllers, better credit assignment, causal intervention analysis, multi-branch search.
7 — Scaling	Test transfer to larger workloads only if earlier evidence supports it.

17. INSTRUCTIONS FOR CLAUDE / BUILDING AI

- Treat this document as a research/product brief, not as proof that the hypothesis works.
- First produce a technical architecture and implementation plan before generating large amounts of code.
- Ask only genuinely blocking questions; otherwise make conservative assumptions and document them.
- Keep controller interfaces vendor-neutral.
- Implement deterministic baselines before intelligent control.
- Make experiments reproducible with seeds and configuration files.

- Log every controller decision and observed outcome.
- Never fabricate benchmark results or claim speedups without measured evidence.
- Keep the UI focused on decisions, experiments, budgets, and evidence rather than decorative AI visuals.
- Write tests for action validation, state construction, memory, experiment branching, rollback, and reproducibility.
- Make it easy to replace Jev with another controller.

18. THE CENTRAL PRINCIPLE

The entire project can be reduced to one principle:

Training becomes a feedback system.

Observe the model. Decide what intervention is worth trying. Execute a bounded training interval. Measure the result. Record the experience. Use that experience to make the next decision. Allocate compute where the expected value is highest. Stop spending compute where the evidence says it is no longer useful.

The ambition is not simply to train models faster. It is to investigate whether training itself can become an adaptive process that learns how to train.

19. RESEARCH NOVELTY CAVEAT

Existing research already explores adaptive data selection, curriculum learning, optimizers, automated hyperparameter optimization, dynamic compute allocation, and related ideas. Therefore, CoreFinity must perform a proper literature review before claiming novelty. The objective is to discover and validate a useful contribution, not to assume that the concept is unprecedented.

A successful outcome can be one of several things: a new algorithm, a new controller architecture, a useful combination of existing techniques, a reproducible empirical finding, or a public framework that makes adaptive-training research substantially easier to conduct.